

LAB # 06

Handwritten Digit Classification using MNIST Dataset

Lab Task:

- Download a dataset containing three input features and three output classes. Load and preprocess the dataset by handling missing values and encoding the target variable for multi-class classification. Build a MLP model using an Ann. Train the model on the dataset and evaluate its performance using appropriate metrics. Visualize the dataset using a three-dimensional (3D) plot to better understand the distribution of the classes and analyze how different features contribute to the classification and examine the decision boundaries of the model.

Code:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.neural_network import MLPClassifier
from mpl_toolkits.mplot3d import Axes3D

df = pd.read_csv("iris.csv")
print(df.head())

X = df.iloc[:, :3].values
y = df.iloc[:, 4].values

X[5, 1] = np.nan
X[20, 2] = np.nan

imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)

encoder = LabelEncoder()
y = encoder.fit_transform(y)

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
model = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    max_iter=1000,
    random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

```

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X[:, 0],
    X[:, 1],
    X[:, 2],
    c=y,
    cmap='viridis',
    s=60
)

ax.set_xlabel('Feature 1')
ax.set_ylabel('Feature 2')
ax.set_zlabel('Feature 3')

plt.title("3D Visualization of Classes")
plt.colorbar(scatter)

plt.show()

X2 = X[:, :2]

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y,
    test_size=0.2,
    random_state=42
)

model2 = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    max_iter=1000,
    random_state=42
)

model2.fit(X2_train, y2_train)

x_min, x_max = X2[:, 0].min() - 1, X2[:, 0].max() + 1
y_min, y_max = X2[:, 1].min() - 1, X2[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)

Z = model2.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.4, cmap='viridis')

plt.scatter(
    X2[:, 0],
    X2[:, 1],
    c=y,
    s=40,
    edgecolors='k',
    cmap='viridis'
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Decision Boundary of ANN Model")

plt.show()

```

Output:-

Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2 Iris-setosa
1	2	4.9	3.0	1.4	0.2 Iris-setosa
2	3	4.7	3.2	1.3	0.2 Iris-setosa
3	4	4.6	3.1	1.5	0.2 Iris-setosa
4	5	5.0	3.6	1.4	0.2 Iris-setosa

Accuracy: 0.16666666666666666				
Classification Report:				
	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.30	1.00	0.46	3
2	0.00	0.00	0.00	3
3	0.00	0.00	0.00	2
6	0.00	0.00	0.00	0
7	0.00	0.00	0.00	1
8	0.00	0.00	0.00	2
9	0.00	0.00	0.00	2
10	0.00	0.00	0.00	1
11	0.00	0.00	0.00	2
12	0.00	0.00	0.00	1
14	0.20	0.50	0.29	2
15	0.00	0.00	0.00	0
16	0.00	0.00	0.00	2
17	0.00	0.00	0.00	1
18	0.00	0.00	0.00	2
19	1.00	0.25	0.40	4
accuracy			0.17	30
macro avg	0.09	0.10	0.07	30
weighted avg	0.18	0.17	0.12	30

